TP — Exploitation d'une API publique

Analyse nutritionnelle + réseau

Objectifs

- Interroger une API REST
- Manipuler une réponse JSON
- Parcourir une liste de dictionnaires
- Générer dynamiquement du HTML
- Ajouter statistiques + couleurs dynamiques
- Séparer logique Python / présentation HTML
- **Analyser les temps de réponse et la taille des données** (mini partie réseau)

📁 Organisation du TP

```
tp_api_alimentation/
│
├── produits.py
├── template.html
└── TP_nutri-score.pdf
```

1. Appel API

Vous devez développer un mini-dashboard affichant des informations nutritionnelles issues d'une API publique.

API utilisée :

```
https://world.openfoodfacts.org/cgi/search.pl
```

Exemple : recherche de biscuits

```
https://world.openfoodfacts.org/cgi/search.pl?
search_terms=biscuits&search_simple=1&action=process&json=1&page_size=10
```

> `page_size=10` limite à 10 produits.

- Limitez `page_size` à 10 ou 20 pour éviter des temps trop longs.
- Limitez le nombre de requêtes.

2. Structure JSON

Ouvrir ce lien dans un navigateur (Firefox) :

https://world.openfoodfacts.org/cgi/search.pl?search_terms=biscuits&search_simple=1&action=process&json=1&page_size=10

Les produits sont dans :

```
data["products"]
```

Chaque produit contient par exemple :

```
{
    "product_name": "...",
    "brands": "...",
    "nutriscore_grade": "a",
    "countries": "...",
}
```

- Nom (`product_name`)
- Marque (`brands`)
- Nutri-Score (`nutriscore_grade`)

> Certaines clés peuvent être absentes → gestion sécurisée nécessaire.

3. Script Python

Récupère pour chaque produit :

- Nom (`product_name`)
- Marque (`brands`)
- Nutri-Score (`nutriscore_grade`)

Crée un fichier `resultat.html` contenant :

- Une liste des produits
- Une couleur selon le Nutri-Score :
  - A → vert
  - B → vert clair
  - C → orange
  - D → rouge orangé
  - E → rouge
  - Non renseigné → gris

Affiche les statistiques

- Nombre total de produits

- Nombre de produits avec Nutri-Score
- Pourcentage correspondant

produits.py

```
import requests

URL = "https://world.openfoodfacts.org/cgi/search.pl?
search_terms=biscuits&search_simple=1&action=process&json=1&page_size=10"

response = requests.get(URL)

if response.status_code != 200:
    print("Erreur API")
    exit()

data = response.json()

produits = data["products"]

liste_html = ""
total_produits = 0
produits_avec_score = 0

for produit in produits:
    nom = produit.get("product_name", "Nom inconnu")
    marque = produit.get("brands", "Marque inconnue")
    nutriscore = produit.get("nutriscore_grade", "Non renseigné")

    total_produits += 1

    # Couleur dynamique selon Nutri-Score
    couleurs = {
        "a": "green",
        "b": "lime",
        "c": "orange",
        "d": "orangered",
        "e": "red"
    }

    couleur = couleurs.get(nutriscore, "gray")

    if nutriscore != "Non renseigné":
        produits_avec_score += 1

    liste_html += f"""
    <li style="color:{couleur}">
        <strong>{nom}</strong><br>
        Marque : {marque}<br>
        Nutri-Score : {nutriscore.upper()}
    </li>
    """
```

```python
# Statistiques
pourcentage_scores = (produits_avec_score / total_produits) * 100

with open("template.html", "r", encoding="utf-8") as f:
    template = f.read()

resultat = template.replace("{{LISTE_PRODUITS}}", liste_html)
resultat = resultat.replace("{{TOTAL}}", str(total_produits))
resultat = resultat.replace("{{POURCENTAGE}}", f"
{pourcentage_scores:.1f}")

with open("resultat.html", "w", encoding="utf-8") as f:
    f.write(resultat)

print("Dashboard généré : resultat.html")
```

4. Template HTML

template.html

```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Dashboard Nutrition</title>
</head>
<body>

<h1>Analyse de produits alimentaires</h1>

<h2>Statistiques</h2>
<p>Nombre total de produits : {{TOTAL}}</p>
<p>Pourcentage avec Nutri-Score : {{POURCENTAGE}} %</p>

<h2>Liste des produits</h2>
<ul>
    {{LISTE_PRODUITS}}
</ul>

</body>
</html>
```

---

5. Analyse réseau de la requête API

Vous devez analyser le comportement réseau de votre requête HTTP.

Objectifs

- Comprendre le rôle du protocole HTTP
- Observer les temps de réponse
- Interpréter les codes de statut HTTP
- Analyser la taille des données transférées
- Comprendre l'impact du volume de données

5.1. Mesurer le temps de réponse

Utiliser le module `time` pour mesurer :

- le temps avant la requête
- le temps après la réponse
- calculer la durée totale

Afficher le temps en secondes.

5.2. Afficher les informations suivantes :

- Code HTTP (`response.status_code`)
- Taille de la réponse en octets
- Type de contenu (`Content-Type`)

5.3. Comparer deux requêtes :

Tester avec :

- `page_size=10`
- `page_size=50`

Comparer :

- temps de réponse
- taille des données

```
import time
import requests

# Mesure du temps
debut = time.time()

URL = "https://world.openfoodfacts.org/cgi/search.pl?
search_terms=biscuits&search_simple=1&action=process&json=1&page_size=10"

response = requests.get(URL, timeout=10)

fin = time.time()
duree = fin - debut

print("Code HTTP :", response.status_code)
print("Temps de réponse : {:.3f} secondes".format(duree))
```

```
print("Taille réponse :", len(response.content), "octets")
print("Type de contenu :", response.headers.get("Content-Type"))
```

Remarques

- Le temps augmente quand page_size augmente ?

  → Plus de données à générer et transférer.

- Que signifie HTTP 200

  → Succès.

- Taille de réponse en octets

  → Volume réel de données réseau.

- Mettre un timeout

  → Éviter blocage.

5.4. Observer la requête avec Wireshark

Filtre :

```
http
```

ou

```
tcp.port == 443
```

- résolution DNS
- handshake TCP
- TLS
- requête GET

5.5. Tester la requête en ligne de commande

```
curl -I https://world.openfoodfacts.org
```

ou

```
curl -w "%{time_total}\n" -o /dev/null -s
"https://world.openfoodfacts.org"
```

Relier Python ↔ outils réseau.